Memory Management

Virtual Memory Management

Lecture 9

Overview

- **Virtual Memory:**
  - Motivation
  - Basic Idea
  - Page Fault
- **Common application of Virtual Memory:**
  - Demand Paging
- **Aspects of virtual memory management:**
  - Page Table Structure
  - Page Replacement Algorithms
  - Frame Allocation

Virtual Memory: Motivation

- Our last assumption of memory usage:
  - Physical memory is large enough to hold one or more process logical memory space completely

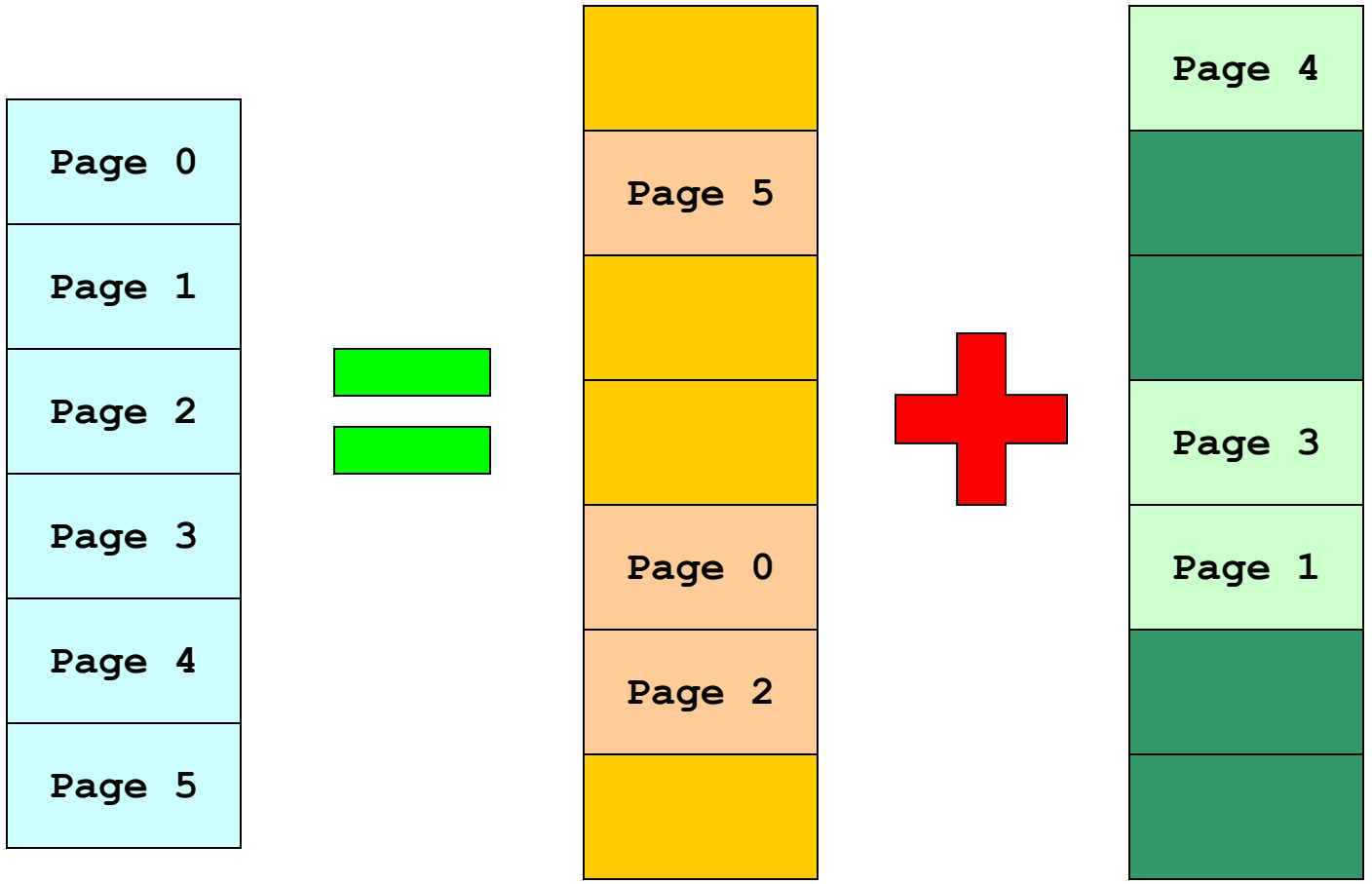
- This assumption is too restrictive:
  - What if the logical memory space of process is >> then physical memory?

  - What if the same program is executed on a computer with lesser physical memory?

Virtual Memory: Basic Idea

- Observation:
  - Secondary storage has much larger capacity compared to physical memory
- Basic Idea:
  - Split the logical address space into small chunks:
    - Some chunks reside in physical memory
    - **Other are stored on secondary storage**
- The most popular approach:
  - Extension of the paging scheme in last lecture:
    - Logical memory space split into fixed size page
    - Some pages may be in physical memory
    - Other in secondary storage

Virtual Memory: Paging Illustration



Logical Memory Space
(Virtual Memory Space)

physical memory

Secondary Storage

Extension of basic paging scheme, i.e. the "virtual" part

Extended Paging Scheme

- Basic idea remains unchanged:
  - Use page table to translate **virtual** address to physical address
- New addition:
  - To distinguish between two page types
    - **memory resident** (pages in physical memory)
    - non-memory resident (pages in secondary storage)
    - ➔ Use a (***is memory resident?***) bit in page table entry
  - CPU can only access memory resident pages:
    - **Page Fault**: When CPU tries to access non-memory resident page
    - OS need to bring a non-memory resident page into physical memory

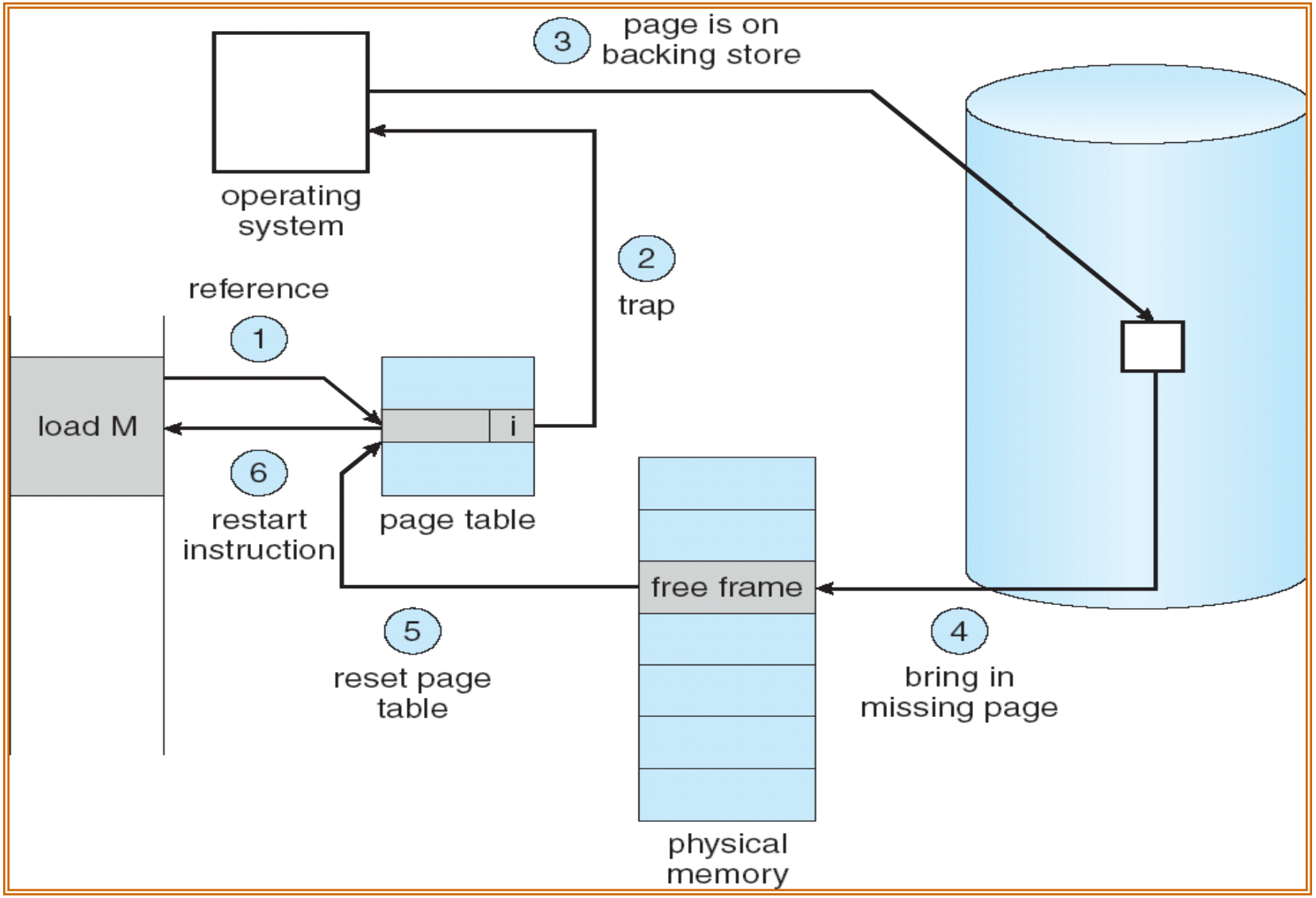
Accessing Page X: General Steps

By Hardware

By OS

1. Check **page table**:
   - Is page **X *memory resident***?
     - Yes: Access physical memory location. Done.
     - No: Continue to the next step
2. **Page Fault: Trap** to OS (OS is informed)
3. Locate page X in secondary storage
4. Load page X into a physical memory frame
5. Update page table
6. Go to step 1 to retry

Virtual Memory Accessing: Illustration



Virtual Memory: Justification

- Observation:
  - Secondary Storage access time >> Physical memory access time
- If memory access results in page fault most of the time:
  - Non-memory resident pages need to be loaded
  - Known as **thrashing**
- How do we know that **thrashing** is **unlikely** to happen?
  - Related: How do we know that after a page is loaded, it is likely to be useful for future accesses?

Recap: **Locality Principles**

- *Most programs* exhibit these behaviors:
  - Most time are spent on a relatively small part of code only
  - In a period of time, accesses are made to a relatively small part of data only
- Formalized as **locality principles**:
  - **Temporal Locality**:
    - Memory address which is used ***is likely to be used again***
  - **Spatial Locality**:
    - Memory addresses close to a used address is likely to be used

Virtual Memory and Locality Principle

- Exploiting **Temporal Locality:**
  - After a page is loaded to physical memory, it is likely to be accessed in near future
    - Cost of loading page is **amortized**
- Exploiting **Spatial Locality:**
  - A page contains contiguous locations that is likely to be accessed in near future
    - Later access to nearby locations will not cause page fault
- However, there are always exceptions ☺
  - Programs that behave badly due to poor design or with malicious intention

Virtual Memory: Summary

- Completely separate logical memory address from physical memory
  - Amount of physical memory no longer restrict the size of logical memory address
- More efficient use of physical memory
  - Page currently not needed can be on secondary storage
- Allow more processes to reside in memory
  - Improve CPU utilization as there are more processes to choose to run

Hmm…. let's use VM to handle this problem…

COMMON APPLICATION

Motivation

- When a new process is launched, should **ALL** text (instruction), data (global variables etc) pages be allocated in RAM?

- Considerations:

1. Large startup cost if there are large number of pages to allocate and initialize
   - But, how do we know which pages to load if we don't load all?
2. Need to reduce the footprint of process in physical memory so that more processes can be memory resident

Demand Paging

- Simple idea:
  - Process start with **no memory resident page**
  - Only allocate a page when there is a **page fault**

- **Pros:**
  - Fast startup time for new process
  - Small memory footprint

- **Cons**:
  - Process may appear sluggish at the start due to multiple page faults
  - The page faults may have cascading effect on other processes (i.e. thrashing)

More on Virtual Memory Management

- More in-depth looks on several aspects:
  - Huge page table with large logical memory space
    ➔ How to structure the page table for efficiency?
    - **Page Table Structures**
  - Each process has limited number of resident memory pages
    ➔ Which page should be replaced when needed?
    - **Page Replacement Algorithms**
  - Limited physical memory frames
    ➔ How to distribute among the processes?
    - **Frame Allocation Policies**

Waste not, want not

PAGE TABLE STRUCTURE

Page Table Structure

- Page table information is kept with the process information and takes up physical memory space

- Modern computer systems provide huge logical memory space
  - 4GiB(32bit) is the norm, 8TiB or more is possible now
  - Huge logical memory space ➔ Huge number of pages
  - ➔ Each page has a page table entry ➔ Huge page table

- Problems with huge page table
  - High overhead
  - Fragmented page table:
    - Page table occupies several memory pages

Page Table Structure: **Direct Paging**

- Direct Paging: keep all entries in a single table

- With 2^p pages in logical memory space
  - **p** bits to specify one unique page
  - 2^p page table entries (PTE), each contains:
    - physical frame number
    - additional information bits (valid/invalid, access right etc)

- Example:
  - Virtual Address: 32 bits, Page Size = 4KiB
  - P = 32 – 12 = 20
  - Size of PTE = 2 bytes
  - Page Table Size = 2^{20} * 2 bytes = 2MiB (!)

2-Level Paging: Basic Idea

- Observation:
  - Not all process uses the full range of virtual memory space ➔ Full page table is a waste!

- Basic Idea:
  - Split the full page table into regions
  - Only a few regions are used
    - As memory usage grows, new region can be allocated
  - This idea is similar to split logical memory space into pages ☺
  - Need a directory to keep track of the regions
    - Analogues of page table ➔ pages

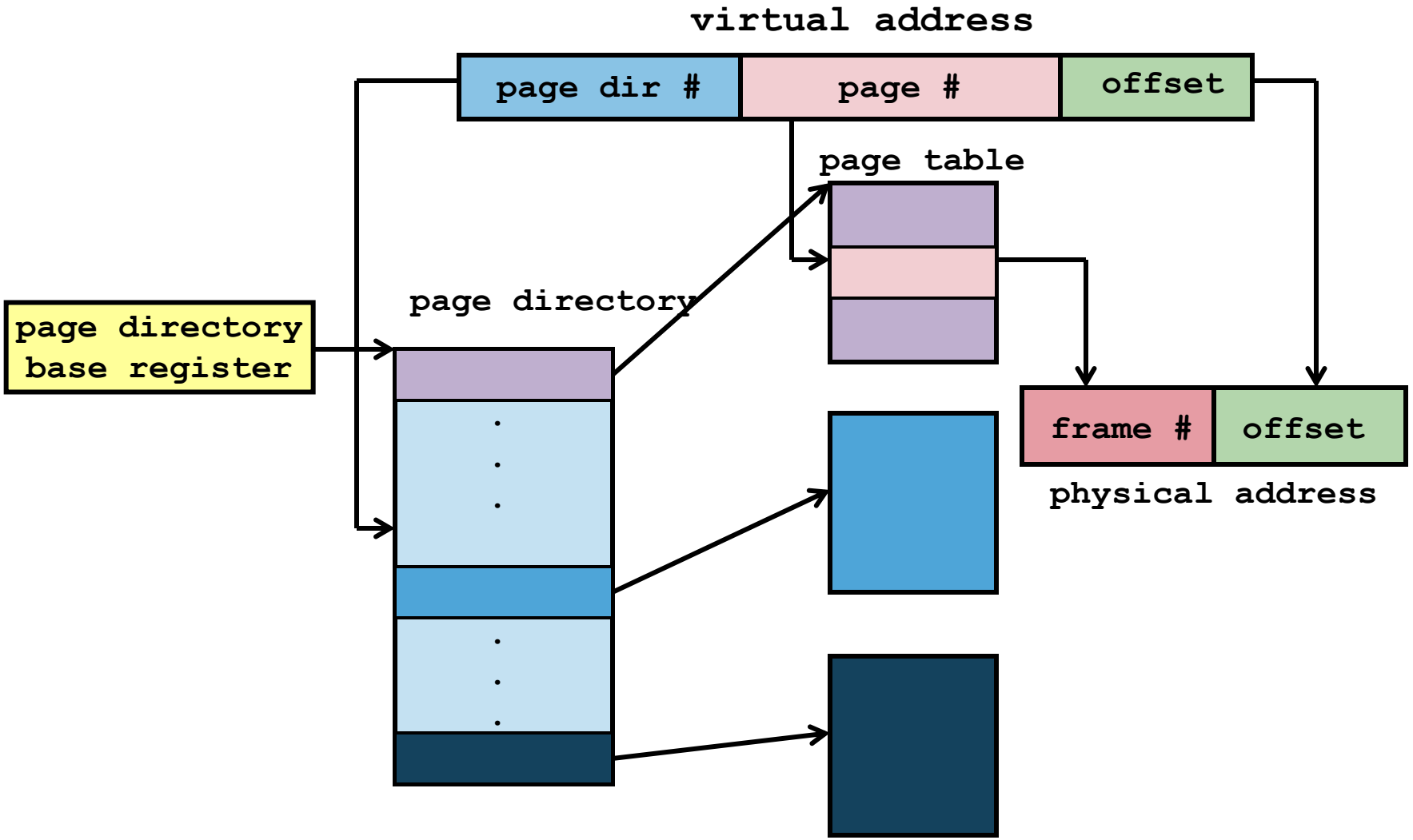
2-Level Paging: Description

- Split page table into *smaller page tables*
  - Each with a **page table number**

- If the original page table has 2^P entries:
  - With 2^M smaller page tables, M bits is needed to uniquely identify one page table
  - Each smaller page table contains $2^{(P-M)}$ entries

- To keep track of the smaller page tables
  - A single **page directory** is needed
  - Page directory contains 2^M indices to locate each of the smaller page table

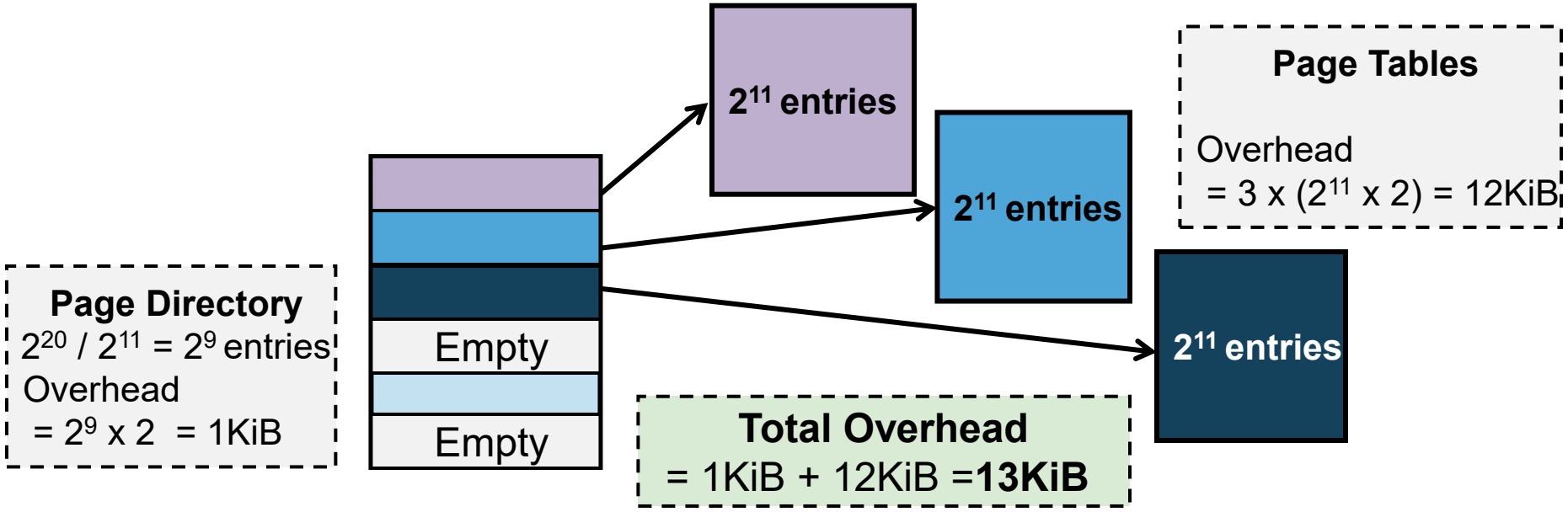
2-level Paging: Illustration



2-Level Paging: **Advantages**

- We can have empty entries in the page directory
  - ➔ The corresponding page tables need not be allocated!

- Using the same setting as the previous example:
  - Assume only **3** page tables is in used
  - Overhead = 1 page directory + 3 smaller page tables



Inverted Page Table: Basic Idea

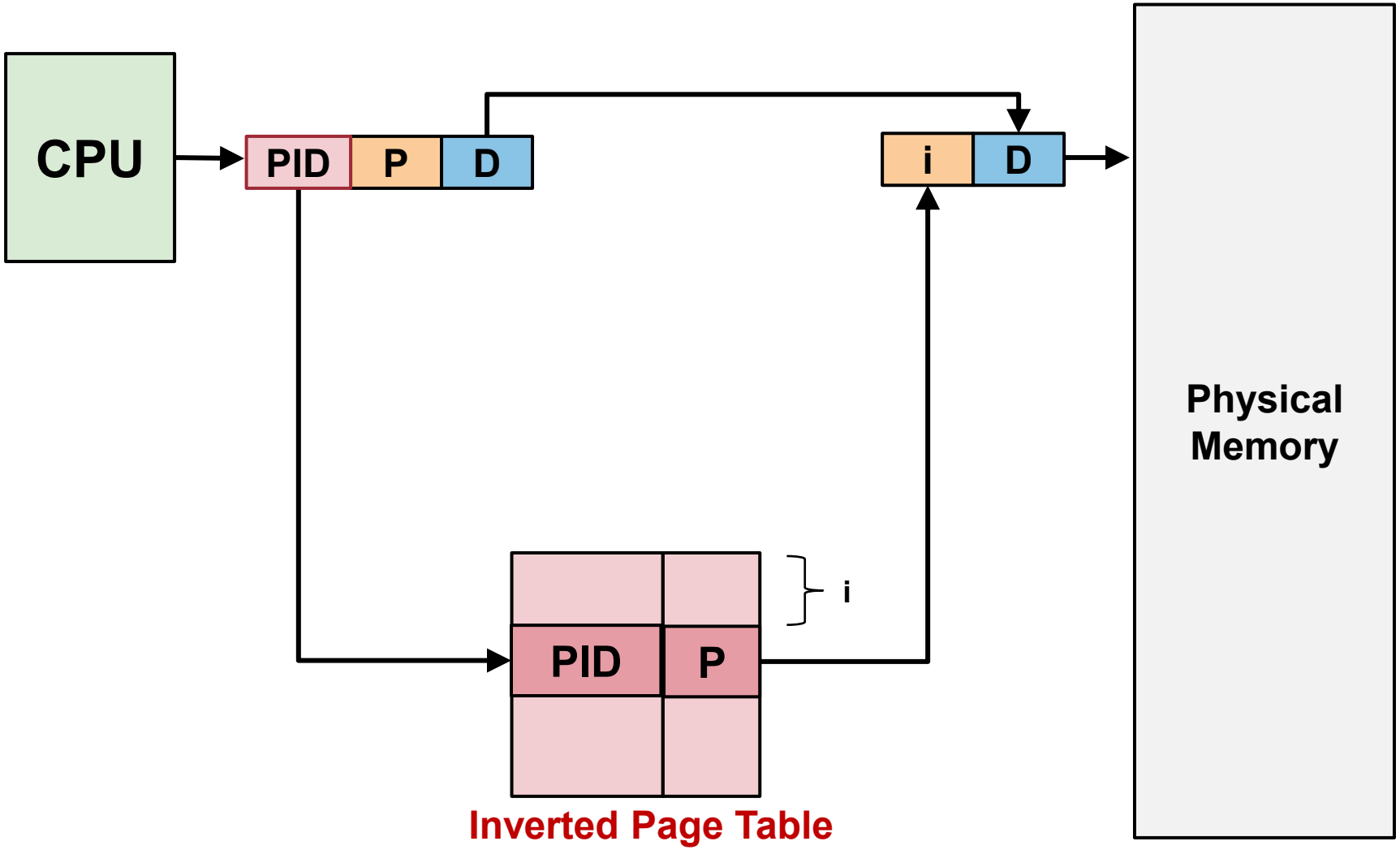
- Page table is a per-process information
  - With M processes in memory, there are M independent page tables
- Observation:
  - Only N physical memory frames can be occupied
  - → Out of the M page tables, only N entries is valid
  - → Huge waste: N << Overhead of M page tables
- Idea:
  - Keep a **single** mapping of physical frame to <pid, page#>
    - pid = process id , page# = page number
    - page# is not unique among processes
    - pid + page# can uniquely identify a memory page

Inverted Page Table: Basic Idea (cont)

- In normal page table, the entries are ordered by page number
  - To lookup page X, simply access the Xth entry
- In inverted page table, the entries are ordered by frame number
  - To lookup page X, need to search the whole table

- **Advantage:**
  - Huge saving: One table for all processes
- **Disadvantage**:
  - Slow translation

Inverted Table: Illustration



Who should I kick out next?

PAGE REPLACEMENT ALGORITHMS

Page Replacement Algorithms

- Suppose there is no free physical memory frame during a page fault:
  - Need to evict (free) a memory page
- When a page is evicted:
  - Clean page: not modified ➔ no need to write back
  - Dirty page: modified ➔ need to write back

- Algorithms to find a suitable replacement page
  - **Optimum (OPT)**
  - **FIFO**
  - **Least Recently Used**
  - **Second-Chance (Clock)**
  - etc…

Modeling Memory References

- In actual memory reference:
  - Logical Address = Page Number + Offset

- However, to study the page replacement algorithm
  - Only **page number** is important

➔ To simplify discussion, memory references are often modeled as **memory reference strings**, i.e. a sequence of page numbers

Page Replacement Algorithms: Evaluation

Memory access time:

$$T_{access} = (1 - p) * T_{mem} + p * T_{page_fault}$$

- p = probability of page fault
- T_{mem} = access time for memory resident page
- T_{page_fault} = access time if page fault occurs

- Since $T_{page_fault} >> T_{mem}$
  - Need to reduce p to keep T_{access} reasonable
- See for yourself, try to find p if:
  - T_{mem} = 100ns, T_{page_fault} = 10ms, T_{access} = 120ns

Good algorithm should **reduce the total number of page faults**

Optimal Page Replacement (OPT)

- General Idea:
  - Replace the page that **will not** be used again for the **longest period of time**
  - **Guarantees** minimum number of page faults
- Unfortunately, not realizable:
  - Need **future knowledge** of memory references
- Still useful:
  - As a base of comparison for other algorithms
  - The closer to OPT == better algorithm

Example: OPT (6 Page Faults)

Time	Memory Reference	Frame			Next Use Time			Fault?
		A	B	C				
1	**2**	**2**			3	∞	∞	**Y**
2	**3**	2	**3**		3	9	∞	**Y**
3	**2**	**2**	3		6	9	∞	
4	**1**	2	3	**1**	6	9	∞	**Y**
5	**5**	2	3	**5**	6	9	8	**Y**
6	**2**	**2**	3	5	10	9	8	
7	**4**	**4**	3	5	∞	9	8	**Y**
8	**5**	4	3	**5**	∞	9	11	
9	**3**	4	**3**	5	∞	∞	11	
10	**2**	**2**	3	5	12	∞	11	**Y**
11	**5**	2	3	**5**	∞	∞	11	
12	**2**	**2**	3	5	∞	∞	∞	

FIFO Page Replacement Algorithm

- General Idea:
  - Memory pages are evicted based on their loading time
  - ➔ Evict the oldest memory page

- Implementation:
  - OS maintain a queue of resident page numbers
    - Remove the first page in queue if replacement is needed
    - Update the queue during page fault trap
  - Simple to implement
    - No hardware support needed

Example: FIFO (9 Page Faults)

Time	Memory Reference	Frame			Loaded at Time			Fault?
		A	B	C				
1	**2**	**2**			1			**Y**
2	**3**	2	**3**		1	2		**Y**
3	**2**	**2**	3		1	2		
4	**1**	2	3	**1**	1	2	4	**Y**
5	**5**	**5**	3	1	5	2	4	**Y**
6	**2**	5	**2**	1	5	6	4	**Y**
7	**4**	5	2	**4**	5	6	7	**Y**
8	**5**	**5**	2	4	5	6	7	
9	**3**	**3**	2	4	9	6	7	**Y**
10	**2**	3	**2**	4	9	6	7	
11	**5**	3	**5**	4	9	11	7	**Y**
12	**2**	3	5	**2**	9	11	12	**Y**

FIFO: Problems

- If physical frame increases (e.g. more RAM)
  - The number page fault should decrease
- FIFO violates this simple intuition!
  - Use 3 / 4 frames to try: **1 2 3 4 1 2 5 1 2 3 4 5**
- Opposite behavior (↑ frames ➔ ↑ page faults)
  - Known as **Belady's Anomaly**
- Reason:
  - FIFO does not exploit **temporal locality**

Least Recently Used Page Replacement (**LRU**)

- General Idea:
  - Make use of temporal locality:
    - Replace the page that has not been used in the longest time
  - Expect a page to be reused in a short time window
    - Have not used for some time ➔ most likely will not be used again
- Notes:
  - Attempts to approximate the OPT algorithm
    - Gives good results generally
  - Does not suffer from Belady's Anomaly

Example: LRU (7 Page Faults)

Time	Memory Reference	Frame			Last Use Time			Fault?
		A	B	C				
1	**2**	**2**			1			**Y**
2	**3**	2	**3**		1	2		**Y**
3	**2**	**2**	3		3	2		
4	**1**	2	3	**1**	3	2	4	**Y**
5	**5**	2	**5**	1	3	5	4	**Y**
6	**2**	**2**	5	1	6	5	4	
7	**4**	2	5	**4**	6	5	7	**Y**
8	**5**	2	**5**	4	6	8	7	
9	**3**	**3**	5	4	9	8	7	**Y**
10	**2**	3	5	**2**	9	8	10	**Y**
11	**5**	3	**5**	2	9	11	10	
12	**2**	3	5	**2**	9	11	12	

LRU: Implementation Details

- Implementing LRU is not easy:
  - Need to keep track of the "last access time" somehow
  - Need substantial hardware support

1. Approach A - **Use a Counter**:
   - A logical "time" counters, which is incremented for every memory reference
   - Page table entry has a "time-of-use" field
     - Store the time counter value whenever reference occurs
     - Replace the page with smallest "time-of-use"
   - Problems:
     - Need to search through all pages
     - "Time-of-use" is forever increasing (overflow possible!)

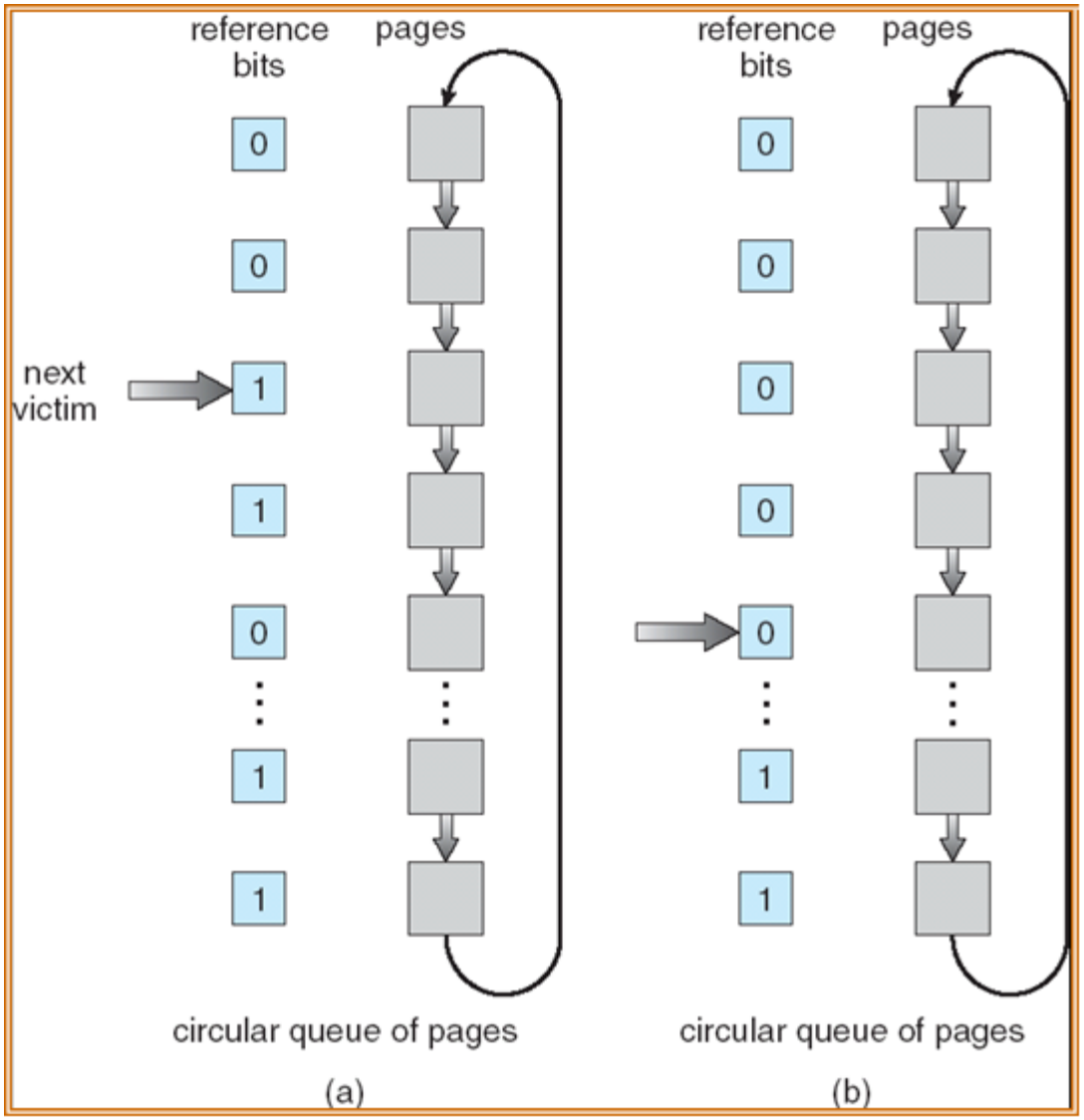
LRU: Implementation Details (cont)

2. Approach B - Use a "Stack":
   - Maintain a stack of page numbers
   - If page **X** is referenced
     - Remove from the stack (for existing entry)
     - Push on top of stack
   - Replace the page at the bottom of stack
     - No need to search through all entries
   - Problems:
     - Not a pure stack: Entries can be removed from any where in the stack
     - Hard to implement in hardware

Second-Chance Page Replacement (CLOCK)

- General Idea:
  - Modified FIFO to give a second chance to pages that are accessed
  - Each PTE now maintains a "reference bit":
    - 1 = Accessed, 0 = Not accessed
  - Algorithm:
    1. The oldest FIFO page is selected
    2. If reference bit == 0 ➔ Page is replaced
    3. If reference bit == 1 ➔ Page is given a 2nd chance
       - Reference bit cleared to 0
       - Arrival time reset ➔ page taken as newly loaded
       - Next FIFO page is selected, go to Step 2
  - Degenerate into FIFO algorithm
    - When all pages has reference bit == 1

Second-Chance: Implementation Details



- Use **circular queue** to maintain the pages:
  - With a pointer pointing to the oldest page (the **victim page**)
- To find a page to be replaced:
  - Advance until a page with '0' reference bit
  - Clear the reference bit as pointer passes through

Example: CLOCK(6 Page Faults)

Time	Memory Reference	Frame (with Ref Bit)			Fault?
		A	B	C	
1	2	▶2 (0)			Y
2	3	▶2 (0)	3 (0)		Y
3	2	▶2 (1)	3 (0)		
4	1	▶2 (1)	3 (0)	1 (0)	Y
5	5	2 (0)	5 (0)	▶1 (0)	Y
6	2	2 (1)	5 (0)	▶1 (0)	
7	4	▶2 (1)	5 (0)	4 (0)	Y
8	5	▶2 (1)	5 (1)	4 (0)	
9	3	2 (0)	5 (0)	▶3 (0)	Y
10	2	2 (1)	5 (0)	▶3 (0)	
11	5	2 (1)	5 (1)	▶3 (0)	
12	2	2 (1)	5 (1)	▶3 (0)	

▶ Victim Page

Which process should I favor?

FRAME ALLOCATION

Frame Allocation

- Consider:
  - There are **N** physical memory frames
  - There are **M** processes competing for frames
  - What is the best way to distribute the **N** frames among **M** processes?

- Simple Approaches:
  - **Equal Allocation:**
    - Each process get **N / M** frames
  - **Proportional Allocation:**
    - Processes are different in size (memory usage)
    - Let $\mathbf{size}_p$ = size of process **p**, $\mathbf{size}_{total}$ = total size of all processes
    - Each process get $\mathbf{size}_p/\mathbf{size}_{total}$***N** frames

Frame Allocation and Page Replacement

- The implicit assumption for page replacement algorithms discussed:
  - Victim page are selected **among pages of the process** that causes page fault
  - Known as **local replacement**

- If victim page can be chosen **among all physical frames**:
  - ➔ Process P can take a frame from Process Q by evicting Q's frame during replacement!
  - Known as **global replacement**

Local vs Global Replacement

- **Local Replacement:**
  - **Pros:**
    - Frames allocated to a process remain constant ➔ Performance is stable between multiple runs
  - **Cons:**
    - If frame allocated is not enough ➔ hinder the progress of a process

- **Global Replacement:**
  - **Pros:**
    - Allow self-adjustment between processes
      - Process that needs more frame can get from other
  - **Cons:**
    - Badly behave process can affect others
    - Frames allocated to a process can be different from run to run

Frame Allocation and Thrashing

- Insufficient physical frame ➔ Thrashing in process
  - Heavy I/O to bring non-resident pages into RAM

- Hard to find the right number of frames:
  - If global replacement is used:
    - A thrashing process "steals" page from other process
    - ➔ cause other process to thrashing (**Cascading Thrashing**)
  - If local replacement is used:
    - Thrashing can be limited to one process
    - But that single process can hog the I/O and degrades the performance of other processes

Finding the right number of frames…

- Observation:
  - The **set** of pages referenced by a process is relatively constant in a period of time
    - Known as **locality**
  - However, as time passes, the set of pages can change

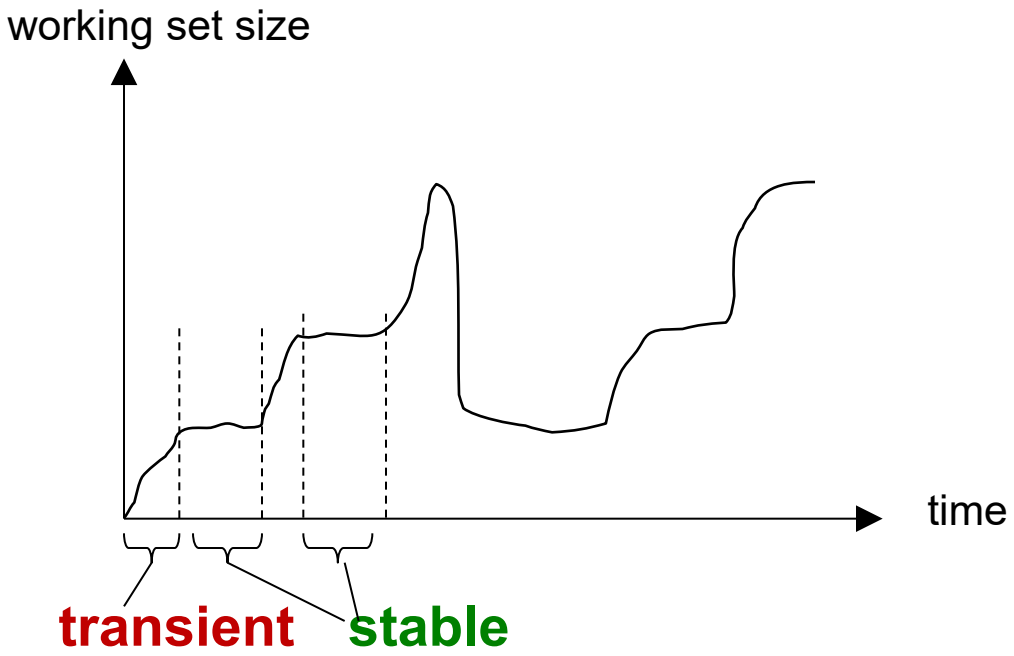
- Example:
  - When a function is executing, the references are likely on:
    - *local variables, parameters, code in that function*
    - these pages define the locality for the function
  - After the function terminates, the references will changes to another set of pages

Working Set Model

- Using the observation on locality:
  - In a new locality:
    - A process will cause page fault for the set of pages
  - With the set of pages in frame:
    - No/few page fault until process transits to new locality

- Working Set Model:
  - Defines Working Set Window Δ
    - An interval of time
  - **`W(t,Δ)`** = active pages in the interval at time **`t`**
  - Allocate enough frames for pages in **`W(t, Δ)`** to reduce possibility of page fault

Working Set Model: Illustration



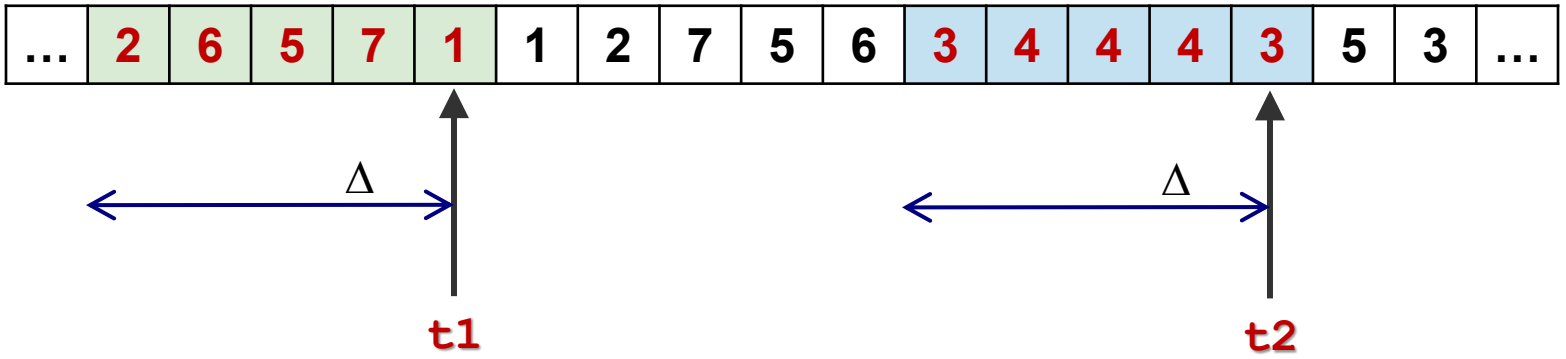
transient region:
working set changing in size

stable region:
working set about the same for a long time

- Accuracy of working set model is directly affected by the choice of Δ
  - Too small: May miss pages in the current locality
  - Too big: May contains pages from different locality

Working Set Model: Illustration

- Example memory reference strings



  - Assume
    - Δ = an interval of 5 memory references
  - **W(*t1*,Δ)={1,2,5,6,7}** (5 frames needed)
  - **W(*t2*,Δ)={3,4}** (2 frames needed)
  - Try using different Δ values

Summary

- Virtual memory
  - The "why" and "how"

- Discussed different aspects of virtual memory management
  - Use different page table structure to reduce page table overhead
  - Use different page replacement algorithms to reduce page fault
  - How frame allocation affects page fault of a process